

Optimización y gestión de *workflows* con APIs

Rate Limits de Gmail API

Cuando un *workflow* usa APIs, cada acción consume recursos. Si el flujo consulta, lee, modifica o envía mensajes muchas veces en poco tiempo, puede alcanzar límites de uso y fallar. Por eso, una solución aplicada **debe controlar cuántas llamadas realiza y con qué frecuencia.**

Cuotas y límites

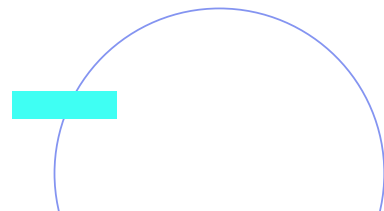
Límites diarios (por proyecto):

- **Envío de *emails*:** 2,000,000,000 unidades/día.

- **Lectura de *emails*:** 1,000,000,000 unidades/día.
- **Modificación de *emails*:** 500,000,000 unidades/día.

Límites por usuario por segundo:

- **250 unidades por usuario/segundo.**
- Permite ráfagas cortas, luego se regula.



Costo en unidades



Operación	Unidades
users.messages.list	5
users.messages.get	5
users.messages.send	100
users.messages.modify	5
users.drafts.create	10
users.drafts.send	100
users.threads.get	10
users.labels.list	1
users.getProfile	1

Estrategias para evitar *Rate Limits*

Batch Processing (Procesamiento por lotes)

Cuando el *workflow* recibe muchos emails al mismo tiempo, procesarlos uno por uno puede generar demoras o alcanzar límites de API. El procesamiento por lotes permite agrupar mensajes y controlar el ritmo de ejecución.

Situación	Riesgo	Solución
Llegan 100 <i>emails</i> juntos	Muchas llamadas seguidas	Procesar en grupos de 10
Cada email requiere IA	Alto consumo de <i>tokens</i> /API	Clasificar primero con reglas
Hay acciones sobre Gmail y Sheets	Saturación de servicios	Agregar pausas entre lotes

Aplicación de *Batch Processing*:

```
// Nodo Code: Batch Processing

const emails = $input.all();

const BATCH_SIZE = 10;

const DELAY_MS = 1000; // 1 segundo entre batches

const batches = [];

for (let i = 0; i < emails.length; i += BATCH_SIZE) {

    batches.push(emails.slice(i, i + BATCH_SIZE));
```



...

```
}  
  
return batches.map((batch, index) => ({  
  json: {  
    batch: batch,  
    batchNumber: index + 1,  
    totalBatches: batches.length,  
    delayMs: DELAY_MS * index  
  }  
}));
```

Exponential Backoff

Cuando una API responde con un error de límite de uso, no conviene repetir la llamada inmediatamente. El *exponential backoff* espera cada vez más tiempo antes de reintentar, para evitar saturar el servicio.

Este patrón mejora la estabilidad del *workflow* y evita que un error temporal interrumpa toda la automatización.

`callApi()` representa la acción del *workflow* que intenta consultar o modificar datos mediante una API. En un flujo real, puede ser una llamada a Gmail, Sheets, Calendar u otro servicio conectado a n8n.

Intento	Tiempo de espera
1	1 segundo
2	2 segundos
3	4 segundos
4	8 segundos
5	16 segundos

Exponential Backoff

```
// Nodo Code: Retry con Exponential Backoff

async function retryWithBackoff(fn, maxRetries = 5) {
  let retries = 0;
  while (retries < maxRetries) {
    try {
      return await fn();
    } catch (error) {
      if (error.status === 429) { // Rate limit exceeded
        const delay = Math.pow(2, retries) * 1000; // 1s, 2s, 4s, 8s, 16s
```


...

```
    console.log(`Rate limit hit. Retrying in ${delay}ms...`);
    await new Promise(resolve => setTimeout(resolve, delay));
    retries++;
  } else {
    throw error;
  }
}
}
throw new Error('Max retries exceeded');
}
// Uso
const result = await retryWithBackoff(async () => {
  return await $('Gmail').execute();
});
return [{ json: result }];
```

Caching con Redis/Database

El caché permite verificar si un *email* ya fue procesado antes de volver a ejecutar acciones sobre él. Esto evita respuestas duplicadas, registros repetidos y llamadas innecesarias a la API.

Ejemplo de clave de caché:

```
email:{{ $json.messageId }}
```

Esta clave combina una etiqueta fija (*email*) con el identificador único del mensaje (*messageId*). Así, cada email procesado queda registrado con una clave propia.

Paso	Qué hace el <i>workflow</i>
1	Recibe un email
2	Busca en cache si existe <code>email:{{ \$json.messageId }}</code>
3	Si existe, no lo procesa de nuevo
4	Si no existe, lo procesa y guarda esa clave en caché



Caching con Redis/Database

```
{  
  "node": "Redis - Check Cache",  
  "type": "n8n-nodes-base.redis",  
  "parameters": {  
    "operation": "get",  
    "key": "email:{{ $json.messageId }}"  
  }  
}
```



Webhooks en lugar de *Polling*

El *polling* revisa una fuente cada cierto tiempo para detectar novedades. Un *webhook*, en cambio, activa el *workflow* cuando ocurre un evento. Elegir uno u otro impacta en consumo de API, velocidad de respuesta y complejidad de implementación.

Criterio	<i>Polling</i>	<i>Webhook</i>
Funcionamiento	Revisa cada cierto intervalo	Se activa ante un evento
Consumo de API	Puede generar consultas repetidas	Reduce consultas innecesarias
Tiempo de respuesta	Depende de la frecuencia de revisión	Más cercano al tiempo real
Complejidad	Más simple de configurar	Requiere configuración adicional
Uso recomendado	Casos simples o de baja frecuencia	Casos críticos o de alto volumen

Webhooks en lugar de *Polling*

Gmail API soporta *push notifications*:

```
# Configurar push notifications (Pub/Sub)

POST https://gmail.googleapis.com/gmail/v1/users/me/watch

{
  "labelIds": ["INBOX"],
  "topicName": "projects/YOUR_PROJECT/topics/gmail-notifications"
}
```

Optimización de recursos

Minimizar Llamadas a API

```
// MALO: Múltiples llamadas individuales
for (const email of emails) {
  const full = await gmail.messages.get({ id: email.id });
  const thread = await gmail.threads.get({ id: email.threadId });
  // Total: 2 llamadas × N emails
}

// BUENO: Batch request
const batch = gmail.newBatch();
emails.forEach(email => {
  batch.add(gmail.messages.get({ id: email.id }));
});
const results = await batch.execute();
// Total: 1 batch request
```

Usar *format parameter*

Gmail API permite diferentes niveles de detalle:

```
// MÍNIMO (1 quota unit): Solo metadata  
  
gmail.messages.get({  
  id: messageId,  
  format: 'minimal'  
});  
// METADATOS (5 quota units): Headers + metadata  
  
gmail.messages.get({  
  id: messageId,
```

...

```
format: 'metadata',
metadataHeaders: ['From', 'To', 'Subject', 'Date']
});

// COMPLETO (5 quota units): Todo incluyendo body
gmail.messages.get({
  id: messageId,
  format: 'full'
});

// RAW (5 quota units): Email en formato RFC 2822
gmail.messages.get({
  id: messageId,
  format: 'raw'
});
```


Workflow optimizado

Un *workflow* optimizado reduce llamadas innecesarias, evita reprocesar mensajes y usa IA solo cuando aporta valor.

Flujo sugerido:

1. Gmail Trigger detecta nuevos emails.
2. Se solicitan solo los datos necesarios.
3. El caché verifica si el email ya fue procesado.
4. Se filtran mensajes repetidos.
5. Los casos simples se clasifican por reglas.
6. Los casos ambiguos se envían a IA.
7. El procesamiento se realiza por lotes cuando hay alto volumen.

Ejemplo de *workflow* optimizado

```
{
  "workflow": "Email Processing - Optimizado",
  "nodes": [
    {
      "name": "Gmail Trigger",
      "parameters": {
        "pollTimes": {
          "item": [
            { "mode": "custom", "hour": "*/1" }
          ]
        }
      }
    }
  ]
}
```

...

]

},

`"filters": {` `"maxResults": 50``},``"options": {` `"format": "metadata",` `"metadataHeaders": ["From", "To", "Subject", "Date"]``}`

...

...

```
    }  
  },  
  
  {  
    "name": "Redis - Cache Check",  
    "description": "Evita procesar emails duplicados"  
  },  
  
  {  
    "name": "Filter - Solo Nuevos",  
    "description": "Procesa solo si no está en cache"  
  },  
  
  {  
    "name": "Classify - Batch Process",  
    "description": "Clasifica en batches de 10"  
  }  
]  
}
```

Monitoreo y alertas

Implementa monitoreo para detectar problemas:

```
// Nodo Code: Monitor de Quota

const metrics = {

  timestamp: new Date().toISOString(),

  workflow: 'email-automation',

  // Contadores

  emailsProcessed: $input.all().length,

  apiCalls: $input.first().json.apiCallsCount || 0,
```

...

```
errors: $input.first().json.errorCount || 0,
// Tiempos
avgProcessingTime: calculateAverage('processingTimeMs'),
totalDuration: Date.now() - $workflow.startTime,
// Rate limit checks
quotaRemaining: checkQuotaRemaining(),
nearQuotaLimit: checkQuotaRemaining() < 1000
};
// Alerta si estamos cerca del límite
if (metrics.nearQuotaLimit) {
  await $('Slack').send({
    text: '⚠️ ALERTA: Cuota de Gmail API cerca del límite',
    metrics: metrics
  });
}
```

...

...

```
// Guardar métricas  
await $('Google Sheets').append({  
  range: 'Metrics!A:H',  
  values: [Object.values(metrics)]  
});  
return [{ json: metrics }];
```

